

Chapter 7 Linked List

Singly Linked Lists

❑ What is a Singly Linked List?

? Prompt

What is a singly linked list?

✓ Answer

A **singly linked list** is a linear data structure made up of **nodes**, where:

- Each node stores **data (element)**
- Each node stores a **reference (pointer) to the next node**
- The last node points to **None (NULL)**

Unlike arrays, elements are **not stored in contiguous memory**.

❑ What is a Node?

? Prompt

What is a node in a singly linked list?

✓ Answer

A **node** is the basic unit of a linked list. It contains:

1. **Element** → the actual data
2. **Next** → reference to the next node

◆ Node structure:

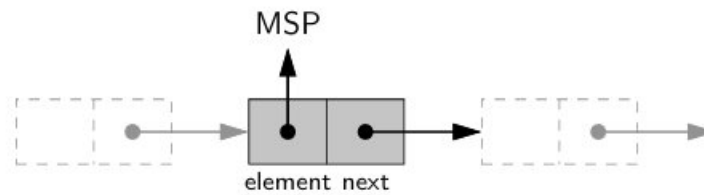


Figure 7.1: Example of a node instance that forms part of a singly linked list. The node's element member references an arbitrary object that is an element of the sequence (the airport code MSP, in this example), while the next member references the subsequent node of the linked list (or None if there is no further node).

Example:

Element: "MSP"

Next: Reference to next node

❏ What are Head and Tail?

? Prompt

What are head and tail in a singly linked list?

✓ Answer

- **Head** → Reference to the **first node**
- **Tail** → Reference to the **last node**
- The tail's `next` pointer is **None**

If head is lost, the entire list becomes inaccessible.

❏ Example of a Singly Linked List

? Prompt

Explain a singly linked list with an example.

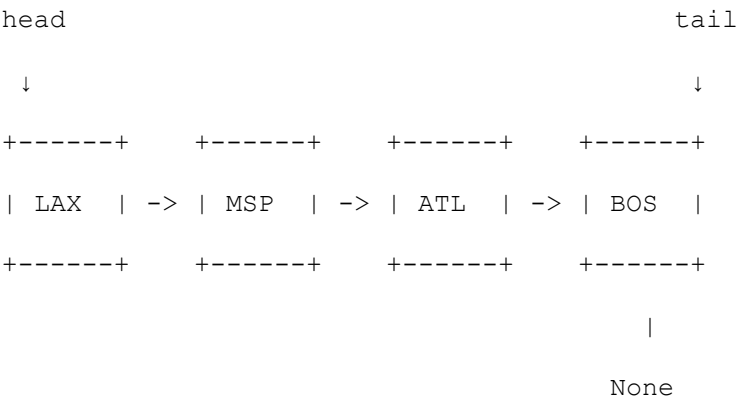
✓ Answer

Consider a list of airport codes:

LAX → MSP → ATL → BOS → None

- Head → LAX
- Tail → BOS
- BOS.next = None

Diagram of Singly Linked List (ASCII Image)



How Does Traversal Work?

? Prompt

What is traversal in a singly linked list?

✓ Answer

Traversal means visiting nodes one by one starting from the **head**, following `next` pointers until `None` is reached.

Also called:

- **Pointer hopping**
- **Link hopping**

↻ Traversal steps:

1. Start at `head`
2. Visit current node
3. Move to `current.next`
4. Stop when `current == None`

❑ Why Store a Tail Reference?

? Prompt

Why do we store a tail reference?

✓ Answer

Without a tail:

- To insert at the end $\rightarrow$ must traverse entire list $\rightarrow O(n)$

With a tail:

- Insert at the end $\rightarrow O(1)$ time

So tail improves **efficiency**.

❑ Why Store the Size of the List?

? Prompt

Why is size stored in a linked list?

✓ Answer

If size is not stored:

- Must traverse list to count nodes $\rightarrow O(n)$

If size is stored:

- Size retrieval $\rightarrow O(1)$

❑ Memory Representation

? Prompt

How is a singly linked list stored in memory?

✓ Answer

- Each node is a **separate object**
- Nodes may be stored **anywhere in memory**
- Linked together using references

This is why linked lists:

- Grow dynamically
- Don't waste memory like arrays

10 Singly Linked List vs Array

? Prompt

Compare singly linked list and array.

✓ Answer

Feature	Array	Singly Linked List
Memory	Contiguous	Non-contiguous
Size	Fixed	Dynamic
Insert/Delete	Slow	Fast
Access	$O(1)$	$O(n)$
Traversal	Index-based	Pointer-based

Insertion in a Singly Linked List

(Head Insertion & Tail Insertion)

◆ PART A: Inserting an Element at the Head

1 What does “inserting at the head” mean?

? Prompt

What is insertion at the head of a singly linked list?

✓ Answer

Insertion at the **head** means adding a new node **at the beginning** of the linked list. The new node becomes the **first node**, and its `next` pointer refers to the **old head**.

This operation is **efficient (O(1))** because no traversal is needed.

2 Why is head insertion easy in a singly linked list?

? Prompt

Why is insertion at the head easy in a singly linked list?

✓ Answer

Because:

- We already have direct access to the **head**
- Only **two pointer changes** are required
- No need to traverse the list

This works even when the list is **empty**.

3 Algorithm: Insert at Head

? Prompt

Write the algorithm to insert an element at the head of a singly linked list.

✓ Answer (Algorithm)

Algorithm `add_first(L, e)`:

1. `newest = Node(e)`
2. `newest.next = L.head`
3. `L.head = newest`
4. `L.size = L.size + 1`

4 Step-by-Step Explanation (Head Insertion)

? Prompt

Explain head insertion step by step.

✓ Answer

1. Create a new node (`newest`)
2. Store the new element in it
3. Set `newest.next` $\rightarrow$ current head
4. Update head $\rightarrow$ `newest`
5. Increase size

⚠ Important:

Always set `newest.next` **before** updating head

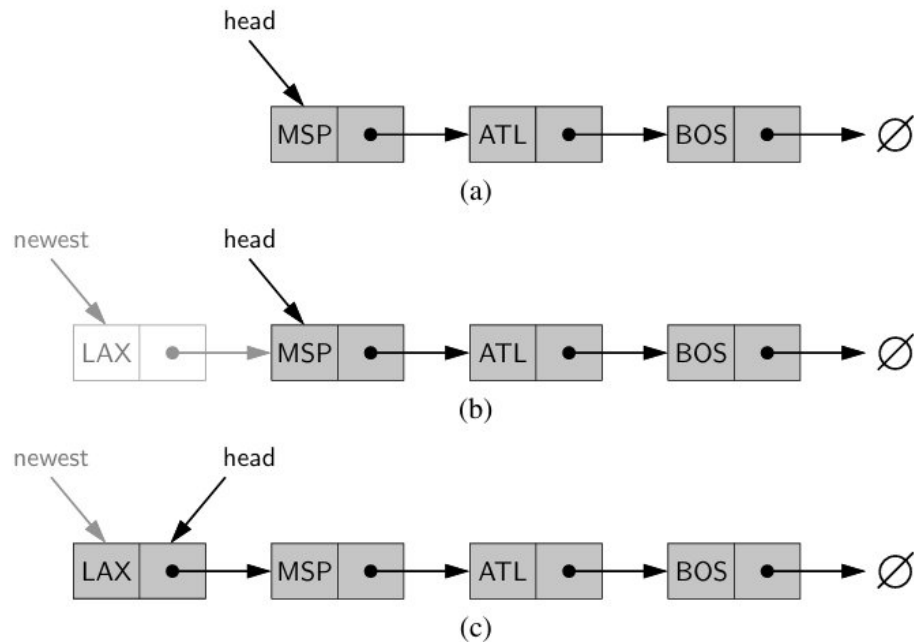


Figure 7.4: Insertion of an element at the head of a singly linked list: (a) before the insertion; (b) after creation of a new node; (c) after reassignment of the head reference.

5 What happens if the list is empty?

? Prompt

What if we insert at the head when the list is empty?

✓ Answer

- `L.head = None`
- `newest.next = None`
- `head` points to the new node
- The new node is both **head** and **tail**

Works naturally — no special case needed!

◆ PART B: Inserting an Element at the Tail

6 What does “inserting at the tail” mean?

? Prompt

What is insertion at the tail of a singly linked list?

✓ Answer

Insertion at the **tail** means adding a new node **at the end** of the list.
The old tail's `next` pointer is updated to point to the new node.

Efficient **only if a tail reference is maintained**.

7 Why do we need a tail reference?

? Prompt

Why is a tail reference needed for tail insertion?

✓ Answer

Without a tail reference:

- Must traverse entire list $\rightarrow O(n)$

With a tail reference:

- Direct access $\rightarrow O(1)$ insertion

8 Algorithm: Insert at Tail

? Prompt

Write the algorithm to insert an element at the tail of a singly linked list.

✓ Answer (Algorithm)

Algorithm `add_last(L, e)`:

1. `newest = Node(e)`
2. `newest.next = None`
3. `L.tail.next = newest`

```
4. L.tail = newest
5. L.size = L.size + 1
```

9 Step-by-Step Explanation (Tail Insertion)

? Prompt

Explain tail insertion step by step.

✓ Answer

1. Create a new node
2. Set its `next` to None
3. Link old tail $\rightarrow$ new node
4. Update tail reference
5. Increment size

⚠ Important:

Set `tail.next` **before** updating `tail`

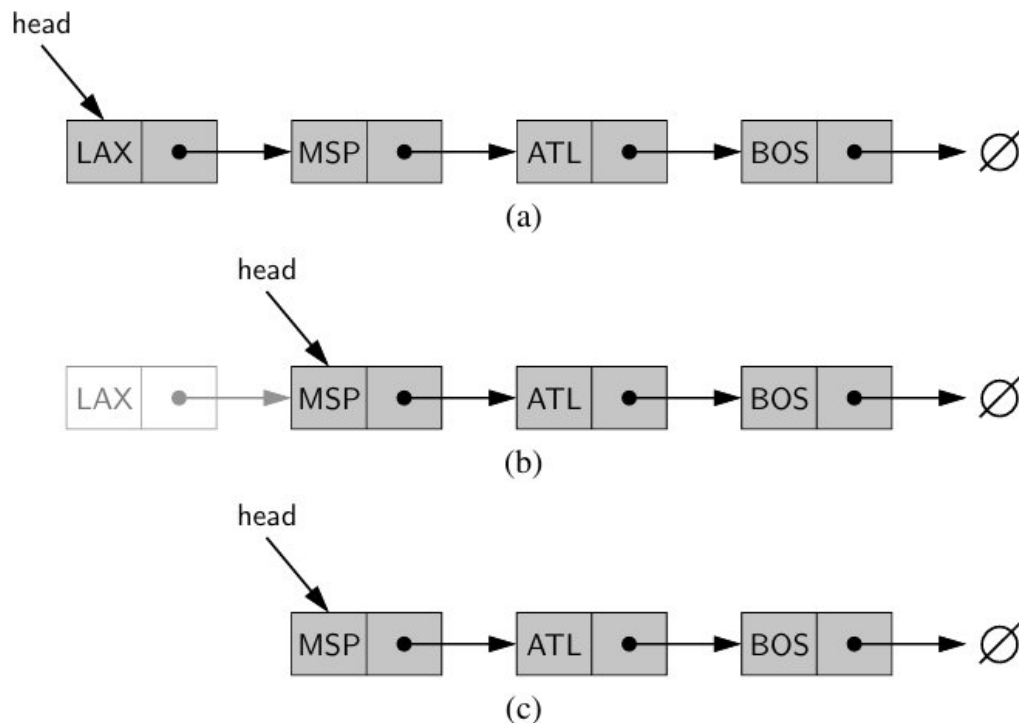


Figure 7.6: Removal of an element at the head of a singly linked list: (a) before the removal; (b) after “linking out” the old head; (c) final configuration.

10 What if the list is empty? (Tail Case)

? Prompt

What adjustment is needed for tail insertion when the list is empty?

✓ Answer

If the list is empty:

- `head = newest`
- `tail = newest`
- `newest.next = None`

Special handling **is required** here.

11 Time Complexity

? Prompt

What is the time complexity of head and tail insertion?

✓ Answer

Operation	Time Complexity
Insert at Head	O(1)
Insert at Tail (with tail)	O(1)
Insert at Tail (no tail)	O(n)

Removal in a Singly Linked List

(Removing from Head & Why Tail Removal is Hard)

◆ PART A: Removing an Element from the Head

? Prompt

What is removal at the head of a singly linked list?

✓ Answer

Removing at the head means deleting the **first node** of the linked list and making the **second node** the new head.

This is the **reverse of head insertion**.

? Prompt

Why is removal at the head easy in a singly linked list?

✓ Answer

Because:

- We already have direct access to the **head**
- Only **one pointer update** is required
- No traversal is needed

This operation is **O(1)**.

? Prompt

Write the algorithm to remove the first node of a singly linked list.

✓ Answer (Algorithm)

```
Algorithm remove_first(L):
```

1. if L.head is None:
 Error: List is empty
2. L.head = L.head.next
3. L.size = L.size - 1

? Prompt

Explain removal at the head step by step.

✓ Answer

1. Check if list is empty
2. Store current head (optional)
3. Move head to next node
4. Old head is now disconnected (garbage collected)
5. Decrease size

Diagram: Remove from Head

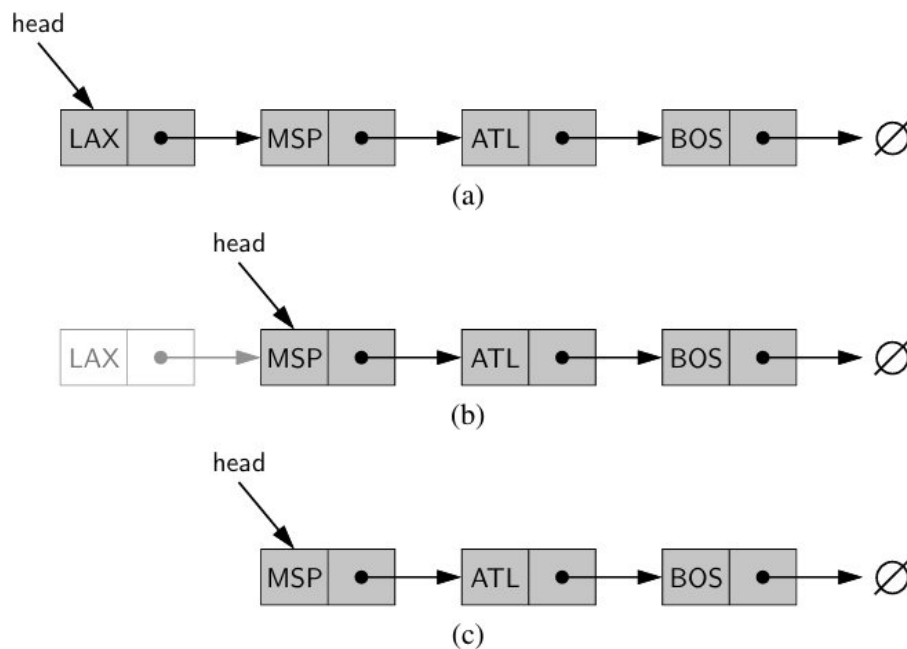


Figure 7.6: Removal of an element at the head of a singly linked list: (a) before the removal; (b) after "linking out" the old head; (c) final configuration.

? Prompt

What happens to the removed head node?

✓ Answer

- The removed node becomes **unreachable**
- It will be cleaned up by **garbage collection**
- In C/C++, it must be **freed manually**

? Prompt

What if the list had only one node?

✓ Answer

If list had one node:

Before:

head → A → None

After:

head → None

List becomes empty.

If tail is maintained:

- tail must also be set to None

◆ PART B: Removing an Element from the Tail

? Prompt

Why is removing the last node difficult in a singly linked list?

✓ Answer

Because:

- Each node only points **forward**
- There is **no pointer to the previous node**
- To delete the last node, we need access to the **second-last node**

But we **cannot go backward**.

? Prompt

How can we remove the last node in a singly linked list?

✓ Answer

We must:

1. Start from the head
2. Traverse the entire list
3. Stop at the second-last node

4. Set its `next = None`
5. Update tail

This takes **O(n)** time.

? Prompt

Write a conceptual algorithm to remove the last node.

✓ Answer (Conceptual)

Algorithm `remove_last(L)`:

1. if `L.head` is `None`:

`Error: List is empty`
2. if `L.head.next` is `None`:

`head = None`

`tail = None`
3. else:

`walk = L.head`

while `walk.next != L.tail`:

`walk = walk.next`

`walk.next = None`

`L.tail = walk`
4. `L.size = L.size - 1`

Diagram: Why Tail Removal Needs Traversal



To remove D, we must find C:

A → B → C → D → None

↑

need this node

We can only reach C by **starting at head**.

❏ Time Complexity Comparison

? Prompt

Compare time complexity of removals.

✓ Answer

Operation	Time
Remove from Head	O(1)
Remove from Tail	O(n)

? Prompt

Why use a doubly linked list for efficient tail removal?

✓ Answer

A **doubly linked list** stores:

- next pointer
- prev pointer

So:

- From tail, we can go to previous node
- Tail removal becomes **O(1)**

Implementing a Stack Using a Singly Linked List

◆ PART A: Conceptual Understanding

? Prompt

What is a stack and which principle does it follow?

✓ Answer

A **stack** is a linear data structure that follows the **LIFO** principle:

Last In, First Out

Operations:

- `push(e)` → insert element
- `pop()` → remove element
- `top()` → view top element
- `isEmpty()` → check if empty
- `size()` → number of elements

? Prompt

Why can a stack be implemented using a singly linked list?

✓ Answer

Because:

- Stack operations affect **only one end** (the top)
- Singly linked lists allow **O(1)** insertion and deletion at the **head**
- No fixed size limitation (dynamic memory)

? Prompt

Should the top of the stack be at the head or tail of the list? Why?

✓ Answer

The **top** of the stack should be at the **head** because:

Location	Insert	Delete
Head	O(1)	O(1)
Tail	O(1)	O(n)

Since stack operations must be efficient, **head is the best choice**.

◆ PART B: Node Design

? Prompt

What does a node store in a linked stack?

✓ Answer

Each node stores:

1. `_element` → actual data
2. `_next` → reference to the next node

? Prompt

Why is `Node` defined as a nested class inside `LinkedStack`?

✓ Answer

Because:

- `Node` is an **implementation detail**
- Users should interact only with stack operations
- Improves **encapsulation and safety**

? Prompt

Why is `__slots__` used in the `Node` class?

✓ Answer

`__slots__`:

- Reduces memory usage
- Prevents creation of unnecessary instance dictionaries
- Useful when **many node objects** are created

◆ PART C: LinkedStack Implementation (Python)

? Prompt

Write the Node class used in a singly linked stack.

✓ Answer (Code)

```
class Node:

    __slots__ = '_element', '_next'

    def __init__(self, element, next):

        self._element = element

        self._next = next
```

? Prompt

What instance variables does LinkedStack maintain?

✓ Answer

Variable	Purpose
head	Reference to top of stack
size	Number of elements

? Prompt

How is an empty stack initialized?

✓ Answer (Code)

```
class LinkedStack:

    class Node:

        __slots__ = '_element', '_next'

        def __init__(self, element, next):

            self._element = element

            self._next = next
```

```
def __init__(self):  
    self._head = None  
    self._size = 0
```

◆ PART D: Stack Operations

? Prompt

How is push implemented using a singly linked list?

✓ Answer

Push is equivalent to **inserting at the head**.

Steps:

1. Create a new node
2. Set its next to current head
3. Update head
4. Increase size

✓ Code

```
def push(self, e):  
    self._head = self.Node(e, self._head)  
    self._size += 1
```

? Prompt

How does the top() method work?

✓ Answer

- Returns the element at the head
- Does NOT remove it
- Raises exception if stack is empty

✓ Code

```
def top(self):  
    if self.is_empty():  
        raise Exception("Stack is empty")  
    return self._head._element
```

? Prompt

How is pop implemented in a linked stack?

✓ Answer

Pop is equivalent to **removing from the head**.

Steps:

1. Save head element
2. Move head to next node
3. Decrease size
4. Return saved element

✓ Code

```
def pop(self):  
    if self.is_empty():  
        raise Exception("Stack is empty")  
    answer = self._head._element  
    self._head = self._head._next  
    self._size -= 1  
    return answer
```

? Prompt

How do we check if the stack is empty and get its size?

✓ Answer (Code)

```
def is_empty(self):  
    return self._size == 0  
  
def __len__(self):  
    return self._size
```

◆ PART E: Performance Analysis

? Prompt

What is the time complexity of `LinkedStack` operations?

✓ Answer

Operation	Time
push	$O(1)$
pop	$O(1)$
top	$O(1)$
size	$O(1)$
isEmpty	$O(1)$

✓ All are worst-case $O(1)$

? Prompt

What is the space complexity of `LinkedStack`?

✓ Answer

- Space usage: $O(n)$
- One node per element

◆ PART F: Comparison with Array Stack

? Prompt

Compare `LinkedList` with `ArrayStack`.

✓ Answer

Feature	<code>ArrayStack</code>	<code>LinkedList</code>
Size	Fixed / Resizable	Dynamic
Push	Amortized $O(1)$	$O(1)$
Pop	$O(1)$	$O(1)$
Memory	Less overhead	Extra pointers

Implementing a Queue Using a Singly Linked List

◆ PART A: Basic Concepts

? Prompt

What is a queue and which principle does it follow?

✓ Answer

A **queue** is a linear data structure that follows the **FIFO principle**:

First In, First Out

Basic operations:

- `enqueue(e)` → insert at rear
- `dequeue()` → remove from front
- `first()` → view front element
- `isEmpty()` → check empty
- `size()` → number of elements

? Prompt

Why can a queue be implemented using a singly linked list?

✓ Answer

Because:

- Insertions are done at **one end (rear)**
- Deletions are done at **the other end (front)**
- Singly linked lists allow efficient operations when both **head and tail** are maintained

All operations can run in **O(1)** time.

? Prompt

Why are both head and tail needed in a linked queue?

✓ Answer

- **Head** → required to dequeue elements (front)
- **Tail** → required to enqueue elements (rear)
- Without tail, enqueue would take **O(n)** time

? Prompt

How are front and rear aligned in a linked queue?

✓ Answer

Queue End	Linked List End
Front	Head
Rear	Tail

This orientation ensures:

- enqueue → insert at tail
- dequeue → remove from head

◆ PART B: Node Design

? Prompt

What information is stored in each node of a linked queue?

✓ Answer

Each node stores:

1. `element` → data
2. `next` → reference to next node

Same node structure as used in **LinkedStack**.

◆ PART C: LinkedList Implementation

? Prompt

What instance variables does LinkedList maintain?

✓ Answer

Variable	Purpose
head	Front of the queue
tail	Rear of the queue
size	Number of elements

? Prompt

How do we initialize an empty linked queue?

✓ Answer (Code)

```
class LinkedList:

    class Node:

        __slots__ = '_element', '_next'

        def __init__(self, element, next):

            self._element = element

            self._next = next

    def __init__(self):

        self._head = None

        self._tail = None

        self._size = 0
```

◆ PART D: Queue Operations

? Prompt

How is enqueue implemented in a singly linked queue?

✓ Answer

Steps:

1. Create a new node
2. Set its `next = None`
3. If queue is empty:
 - `head = newest`
1. Else:
 - `tail.next = newest`
1. Update tail
2. Increase size

✓ Code

```
def enqueue(self, e):  
    newest = self.Node(e, None)  
  
    if self.is_empty():  
        self._head = newest  
  
    else:  
        self._tail._next = newest  
  
    self._tail = newest  
  
    self._size += 1
```

? Prompt

How is dequeue implemented in a linked queue?

✓ Answer

Steps:

1. Check if queue is empty
2. Save head element

3. Move head to next node
4. Decrease size
5. If queue becomes empty:
 - o set tail = None
1. Return saved element

✓ Code

```
def dequeue(self):  
    if self.is_empty():  
        raise Exception("Queue is empty")  
  
    answer = self._head._element  
    self._head = self._head._next  
    self._size -= 1  
  
    if self.is_empty():  
        self._tail = None  
  
    return answer
```

? Prompt

How does the first() method work?

✓ Answer

- Returns element at head
- Does not remove it
- Raises exception if empty

✓ Code

```
def first(self):  
    if self.is_empty():  
        raise Exception("Queue is empty")  
  
    return self._head._element
```

? Prompt

How are isEmpty and size implemented?

✓ Answer (Code)

```
def is_empty(self):  
    return self._size == 0  
  
def __len__(self):  
    return self._size
```

◆ PART E: Special Cases

? Prompt

Why must tail be updated when dequeue removes the last element?

✓ Answer

When queue has only one element:

- head == tail
- Removing head also removes tail
- tail must be set to None to avoid dangling reference

? Prompt

Why does enqueue need special handling when the queue is empty?

✓ Answer

Because:

- The new node is both **head and tail**
- No existing tail exists to link to

◆ PART F: Performance Analysis

? Prompt

What is the time complexity of LinkedList operations?

✓ Answer

Operation	Time
enqueue	$O(1)$
dequeue	$O(1)$
first	$O(1)$
size	$O(1)$
isEmpty	$O(1)$

✓ Worst-case constant time

? Prompt

What is the space complexity of `LinkedList`?

✓ Answer

- Space usage: $O(n)$
- One node per element

◆ PART G: Comparison with Other Queues

? Prompt

Compare linked queue and array-based queue.

✓ Answer

Feature	Array Queue	Linked Queue
Size	Fixed / Resizable	Dynamic
Enqueue	Amortized $O(1)$	$O(1)$
Dequeue	$O(1)$	$O(1)$
Memory	Compact	Extra pointer